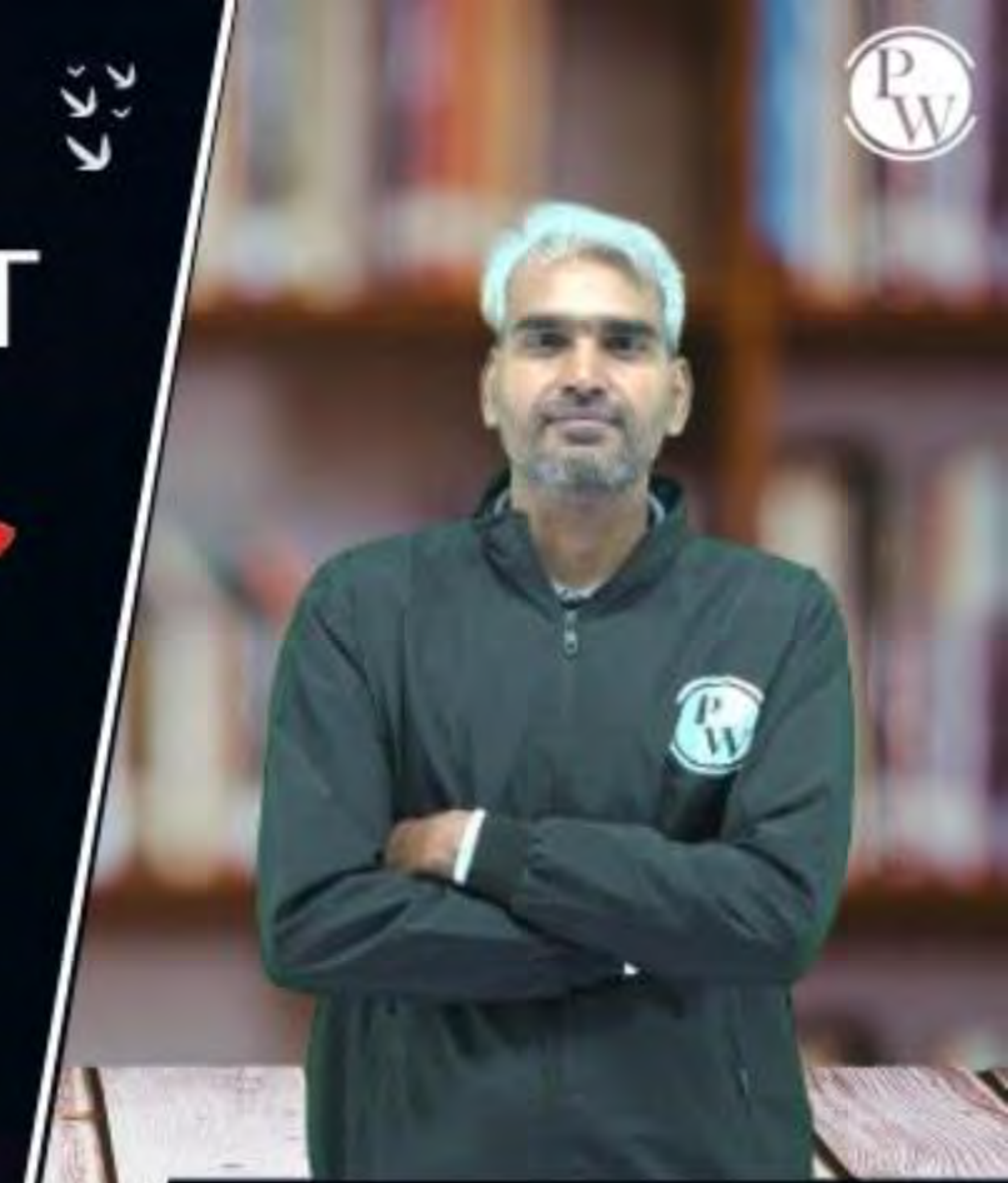
Computer Science & IT

Database Management System

**File organization
And
Indexing**

Lecture No. 06



By- Vishal Sir

Recap of Previous Lecture

Topic Dynamic Multi level index

Topic Structure of B tree

Topic Structure of B+ tree

Topics to be Covered

Topic Insertion in B tree

Topic Deletion from B tree

Slide

Insertion in B tree

Insertion in B tree :→ Insertion will always be Performed at leaf node.

* Let order of a node of B tree = P

1. Search for the leaf node that can hold the key value that needs to be inserted

2. If leaf node is not already full {i.e. number of keys $< (P-1)$} then simply insert the key in the ascending order

3. If leaf node is already full {i.e. no. of keys in leaf node $= (P-1)$} then because of insertion of this new key there will be overflow in that node. In that case we will insert the new key in the leaf node in ascending order, then we will split the node into two parts and we will promote the median position key to the parent node

3 (i) If parent node was not already full, then it will be able to accomodate the Promoted Key, and further action will be required.

3 (ii) If parent node was already full, then split that node into two parts, and promote the median Position Key to the parent node of that node. This may go upto root node, and if root node is also full, then we will split the root node as well into two parts, And the median position Key will become the new root node {i.e; Height of tree will increase by 1}.

Q:- Let order of B tree = 4.
and keys to be inserted are.
90, 10, 2, 8, 30, 94, 14, 28, 38, 20 & 25
in the same order.

* Construct the B tree

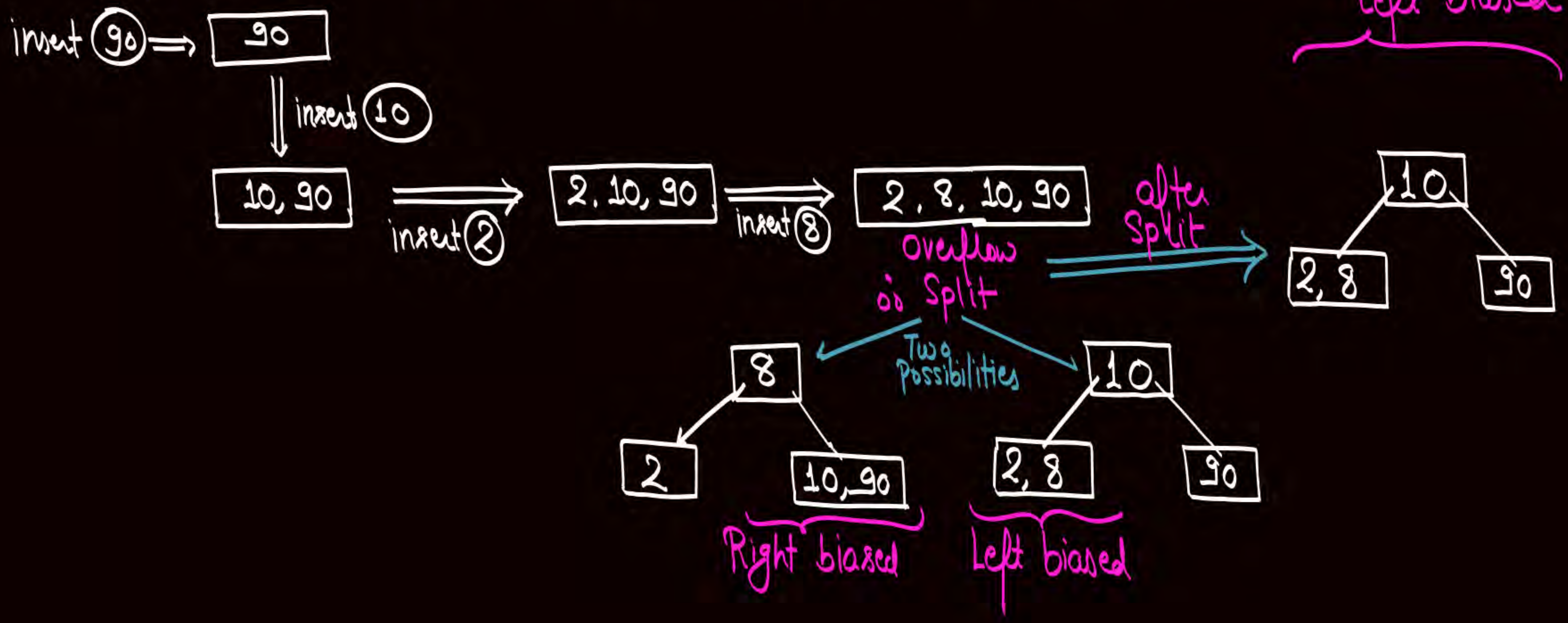
* Order of a node of B tree = 4.
* Maximum number of child Pointer a node can have = 4

∴ Maximum number of Keys a node of B tree can have = 4 - 1 = 3

⇒ Min no. of keys necessary in a non-root node $= \lceil \frac{p}{2} \rceil - 1 = \lceil \frac{4}{2} \rceil - 1 = 1$

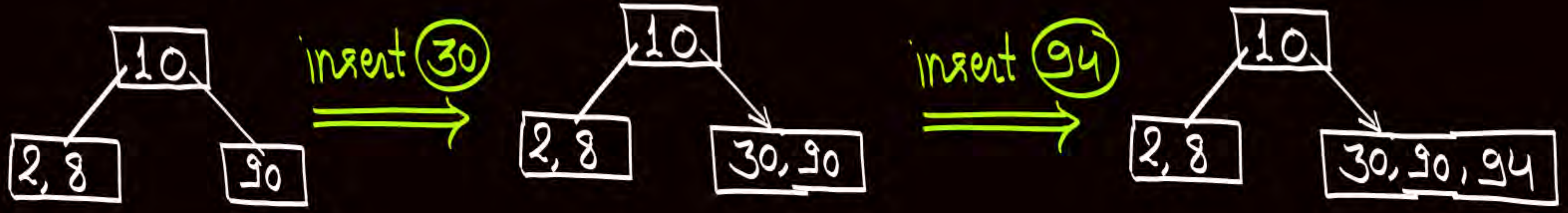
90, 10, 2, 8, 30, 94, 14, 28, 38, 20 & 25

Let us Choose left biased



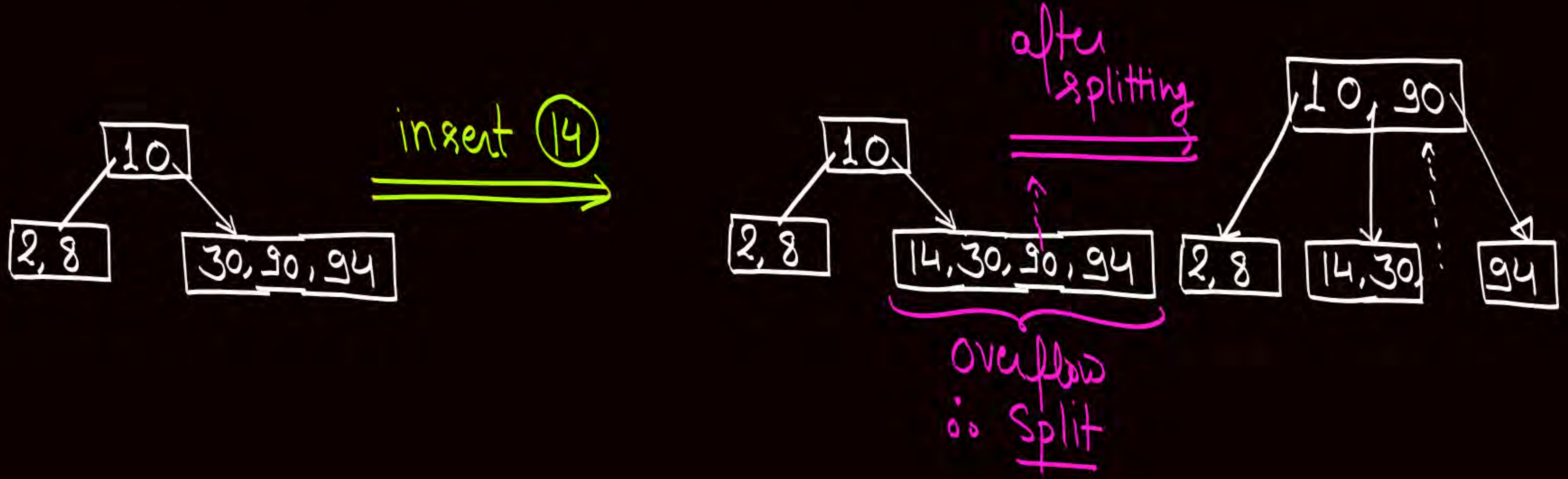
* Order of a node of B tree = 4.
  * Maximum number of child Pointer a node can have = 4
  * Maximum number of Keys a node of B tree can have = 4-1 = 3

90, 10, 2, 8, 30, 94, 14, 28, 38, 20 & 25



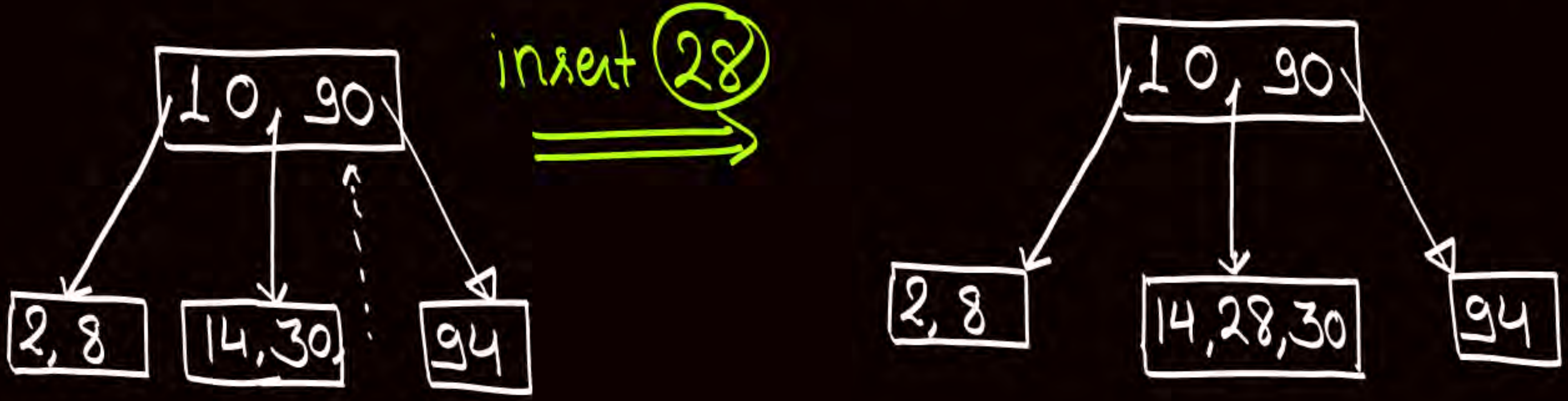
* Order of a node of B tree = 4.
  - Maximum number of child Pointer a node can have = 4
  - ∴ Maximum number of Keys a node of B tree can have = 4-1 = 3

90, 10, 2, 8, 30, 94, 14, 28, 38, 20 & 25



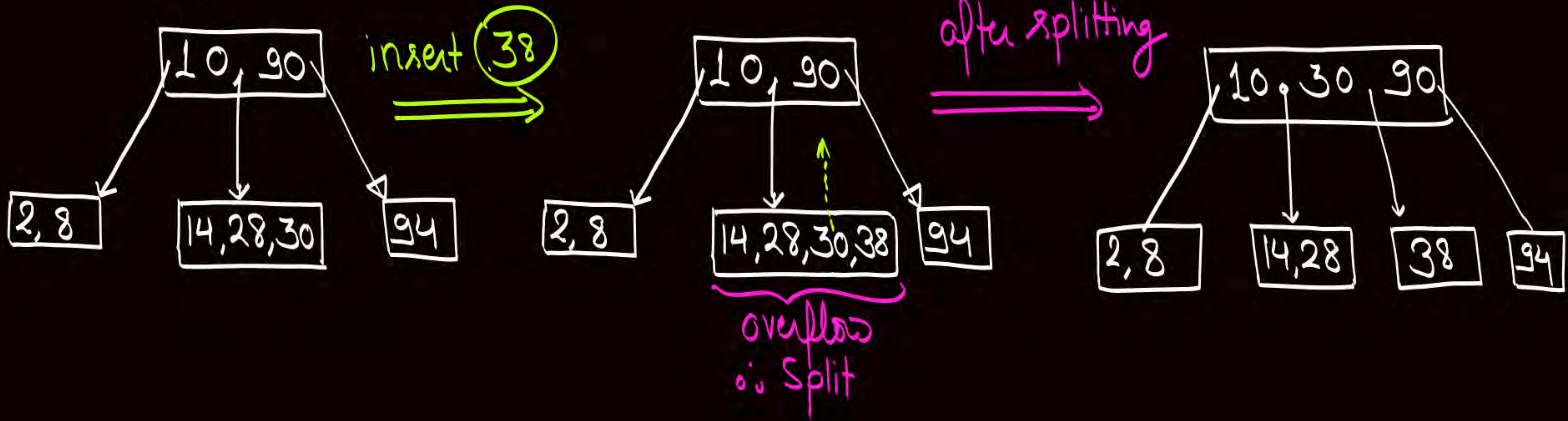
* Order of a node of B tree = 4.
  * Maximum number of child Pointer a node can have = 4
  * Maximum number of Keys a node of B tree can have = 4-1 = 3

90, 10, 2, 8, 30, 94, 14, 28, 38, 20 & 25



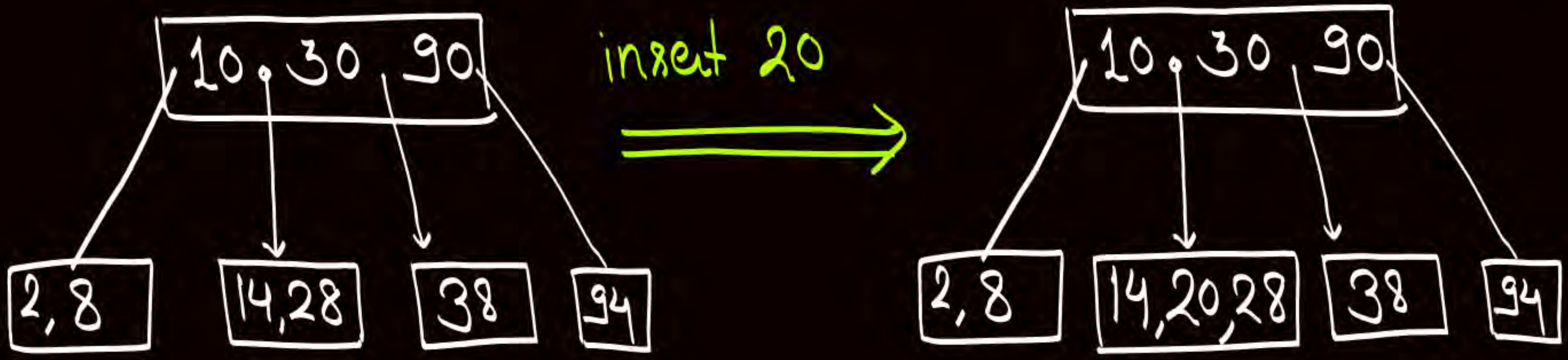
* Order of a node of B tree = 4.
  * Maximum number of Child Pointer a node can have = 4
  * ∴ Maximum number of Keys a node of B tree can have = 4-1 = 3

90, 10, 2, 8, 30, 94, 14, 28, 38, 20 & 25



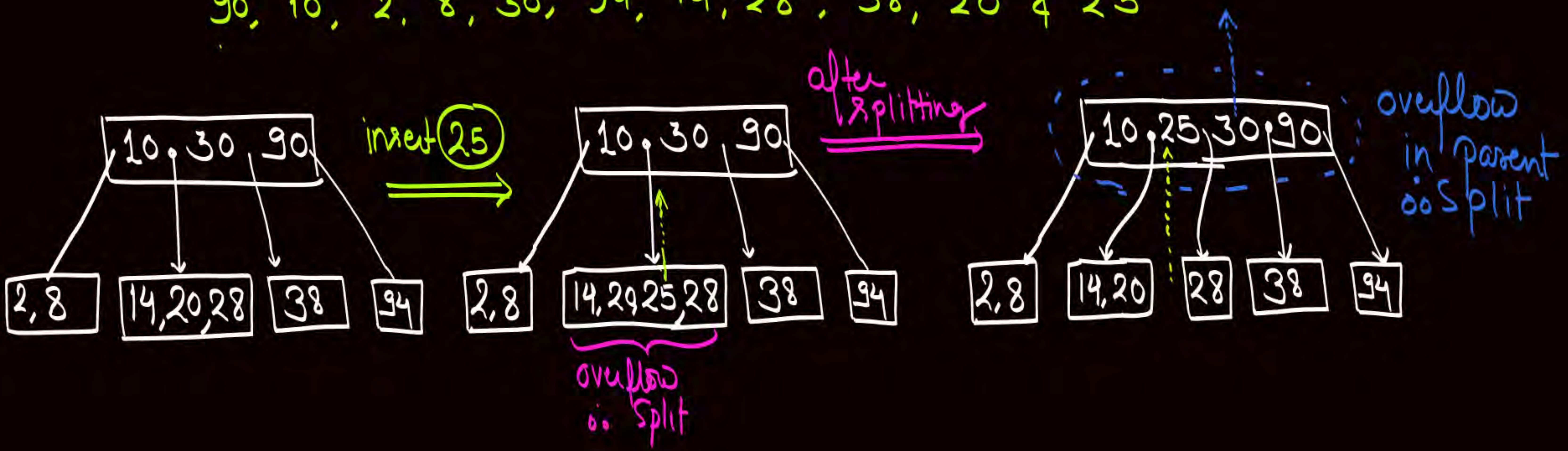
* Order of a node of B tree = 4.
  * Maximum number of Child Pointer a node can have = 4
  * Maximum number of Keys a node of B tree can have = 4-1 = 3

90, 10, 2, 8, 30, 94, 14, 28, 38, 20 & 25



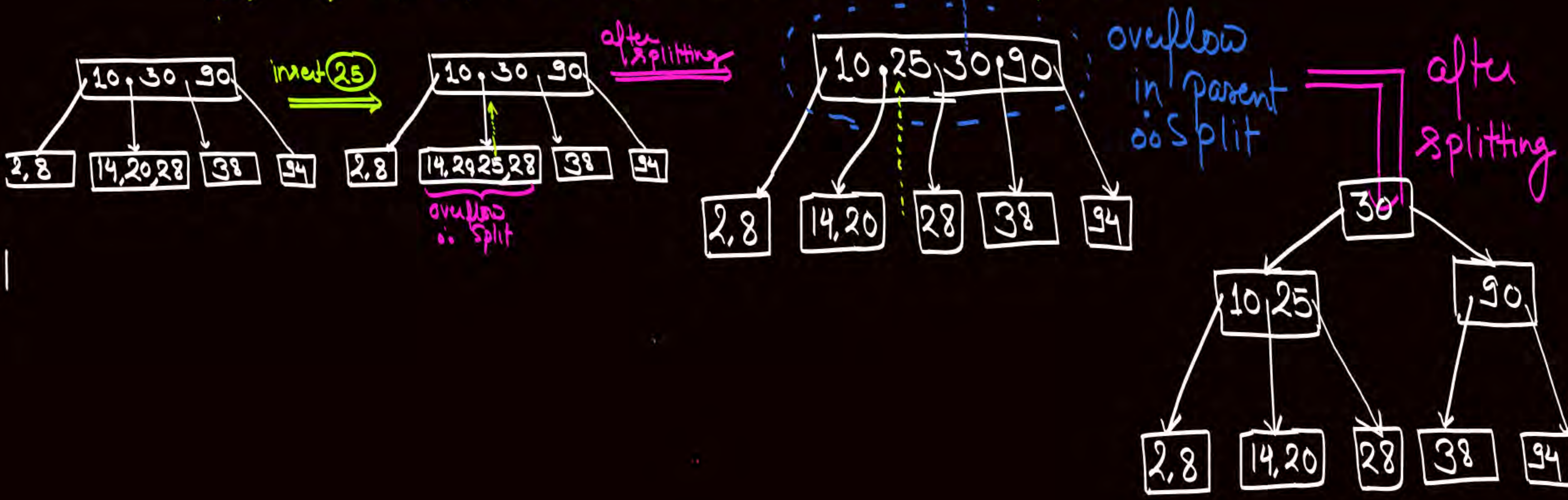
* Order of a node of B tree = 4.
  * Maximum number of child Pointer a node can have = 4
  * Maximum number of Keys a node of B tree can have = 4-1 = 3

90, 10, 2, 8, 30, 94, 14, 28, 38, 20 & 25



* Order of a node of B tree = 4.
∴ Maximum number of child pointer a node can have = 4
∴ Maximum number of keys a node of B tree can have = 4-1 = 3

90, 10, 2, 8, 30, 94, 14, 28, 38, 20 & 25



Deletion from B tree

Deletion from B Tree :-

{ "Key" that needs to be deleted may be in a leaf node or may be in an internal node }

* If key that needs to be deleted is present in internal node of B tree, then replace it by in-order predecessor {i.e. largest key in left sub-tree}

  or

  in-order successor {i.e smallest key from right sub-tree}

  And then delete the key from leaf node

* If key is already at the leaf node delete it from that leaf node

1. After deletion of the key from leaf node if leaf node contains at least the minimumum numer of keys required, then no further action is needed

② After deletion of the key from leaf node if leaf node becomes deficient {i.e. Contains less than the minimum number of keys required}

2(i) Check with your immediate sibling if they can help.
{first check with left sibling, then check with right sibling}

ⓐ If left sibling can help then borrow (re-distribute) from left sibling
Largest key from the left sibling will be promoted to Parent node, and corresponding key from parent {i.e. key b/w this two nodes} will be transferred to deficient node

ⓑ If right sibling can help then borrow (re-distribute) from right sibling
Smallest key from right sibling will be promoted to Parent node, and corresponding key from Parent node {i.e; key between those two nodes} will be transferred to deficient node

2 (ii) If none of your immediate sibling can help, then merge deficient node with one of its sibling along with one key (i.e. the key b/w deficient node & its sibling) from its parent node into a single node., as a result of this process the parent node may become deficient

* If parent node does not become deficient then stop.

* If parent node become deficient, then the same process will be repeated (i) Ask from their sibling if they can help.

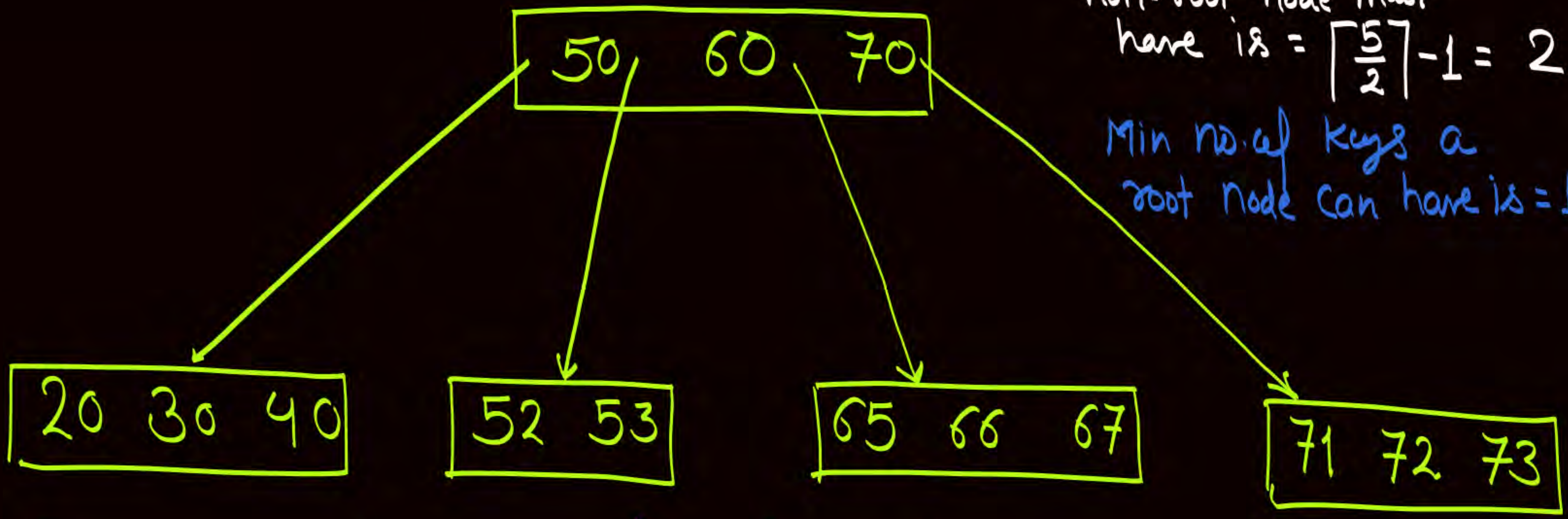
(ii) If not, then merge with one sibling along with one key from parent node

↳ This may go upto root node, if root node becomes deficient then height of tree will reduce by "1".

Q:- Consider the following B tree of order = 5.

Min. no. of keys a non-root node must have is = $\lceil \frac{5}{2} \rceil - 1 = 2$

Min no. of keys a root node can have is = 1



Delete the following keys from the above B tree in the same order

65, 53, 66, 52, 60

2 mins Summary

Topic — Insertion in B tree

Topic — Deletion from B tree

Slide

THANK - YOU

Slide